

Swing Trading Alert Dashboard

Project Plan, Architecture & Continuation Notes

Status as of: June 20, 2026

Scope: Alerts-only decision support. This system never places trades automatically.

Lineage: Fresh build. Not a continuation of, and shares no code with, a prior semi-automated trading bot attempt.

Stack: Python 3.14.5, Windows 11, Cursor, Git, Alpaca (paper account), Pushover.

Purpose of this document: this file is meant to live at the project root. It captures the evidence-based rule set the system is built on, every environment and architecture decision made so far with the reasoning behind it, the current build status, and the exact next steps - so that this conversation, a future session, or a coding agent can pick the project up without re-deriving any of it.

1. Project Overview & Constraints

A personal dashboard that watches a curated list of stocks and crypto assets and pushes **alerts only** (via Pushover) when predefined, rule-based conditions are met - entries, exits, and stop/target levels. It does not connect to any order-execution path. Alpaca is used purely as a free, reliable real-time data source (and its paper account makes it easy to later validate signals against simulated fills if desired - but live order placement is explicitly out of scope for this build).

- **Holding period:** swing trading, days-to-weeks.
- **Markets:** stocks and crypto (BTC/ETH plus a small set of liquid large-cap alts), in one unified dashboard.
- **Hard constraint:** no automated order placement, ever. Alerts are advisory; the human pulls every trigger manually.
- **Relationship to prior project:** this is a fresh codebase. An earlier solo attempt at a semi-automated trading bot stalled after about a month; nothing from that codebase is being reused here.

2. Evidence-Based Rule Set (source of truth for all signal logic)

Distilled from the project's research brief (*Rules for a Personal Swing-Trading Alert Dashboard*). Three pillars carry most of the risk-adjusted benefit; oscillators are confirmation, never standalone triggers. The core trade-off the research supports: trailing stops and volatility-based exits raise Sharpe ratio and cut drawdowns substantially, but typically cost a little raw return versus buy-and-hold - an acceptable trade for a capital-preservation-first, alerts-based system.

2.1 Exit rules, in priority order

- **Tier 1 - ATR trailing stop / Chandelier Exit.** 22-period highest-high minus (ATR x multiplier). Multiplier 3.0 for stocks, widened to 3-5 for crypto. This stop only ever tightens in the trade's favor, never loosens.
- **Tier 1 - Initial structural stop.** Placed at the swing low / consolidation low / moving average the setup is built on. This defines the 1R unit used for position sizing.
- **Tier 1 - Profit target.** 2:1 minimum reward:risk, 3:1 preferred.
- **Tier 2 - Partial scale-out (post-MVP).** e.g. 50% at 1R, 25% at 2R, trail the remainder. Framed correctly: this reduces variance, not expected value per trade.
- **Tier 2 - MA / MACD trend-failure exit (post-MVP).** Close below 20/50 EMA, or a MACD bearish crossover. Prone to whipsaw outside of trends.
- **Tier 2 - Time-based stale-trade exit (post-MVP).** Flag if flat after N days.
- **Tier 3 - RSI / MACD.** Confirmation and divergence only, never a standalone exit trigger (RSI>70 in a strong uptrend is normal, not a sell signal by itself).

2.2 Entry rules, paired with the exits above

- **Gate everything: trend/regime filter.** Only generate long alerts when price is above a rising 50-day AND 200-day moving average. This single filter is the primary defense against the whipsaw failure mode in choppy markets.

- **Setup A - Pullback to MA.** In a confirmed uptrend, pullback to the 20/21 or 50 EMA on declining volume, entry on a bullish reversal candle (engulfing / hammer) or a break of that candle's high. Stop below the swing low.
- **Setup B - Breakout from consolidation.** Daily close above a multi-day/week range on volume at least 1.5-2x average. Stop below the range; target via measured-move projection. Volume is the deciding filter against false breakouts.
- **Post-MVP:** MA crossover entries (regime-filtered), counter-trend oversold bounces at major support, MACD+RSI confluence as an added confirmation layer.

2.3 Risk management

- **Position sizing - fixed-fractional ("1% rule").** Risk per trade = 0.5-1% of current equity (start at 0.5% while proving the system). Size = (equity x risk%) / (entry - stop).
- **Portfolio heat cap.** Total simultaneous open risk capped at 6%. New alerts that would push aggregate risk over the cap are flagged/blocked, not silently allowed.
- **Concentration cap.** No single position over roughly 10-20% of portfolio value, independent of the risk cap.
- **Correlation awareness, crypto especially.** Most altcoins move with BTC at a correlation above 0.7 (BTC-ETH exceeds 0.85 in stress). Treat aggregate "BTC beta" as one risk factor and cap it, rather than treating each alt as independently diversifying.
- **Minimum 2:1 reward:risk** as a hard gate on every alert, no exceptions.

2.4 Stocks vs. crypto calibration

Same underlying logic, different numbers. Never reuse a stock's ATR multiplier on a crypto asset - every stop distance is derived from that asset's own live-computed ATR.

Dimension	Stocks	Crypto
Market hours	~6.5h/day; weekend/overnight gaps	24/7/365; no gaps, but stops stay exposed while you sleep
Typical daily volatility	Large-cap ~1-3%; tech 2-4%	BTC ~2-5%; mid-caps 5-15%; low-caps/memecoins 20-50%
ATR multiplier	2.0-3.0	Often 3-5; widen for alts; always $\geq 1x$ that asset's ATR
Liquidity / spread	Large-caps very tight (~0.1%)	BTC/ETH tight; alts wide (1%+), thin and worst between 00:00-06:00 UTC
Correlation structure	Sector-driven	Most assets correlate to BTC above 0.7
Position sizing	Standard 1% rule	Same rule, but smaller size to hold dollar risk constant against wider stops

- Recompute ATR live per asset; never hard-code a volatility figure.
- Prefer ATR-derived, irregular stop levels over round numbers (round-number stops invite stop-hunting).
- Weight stop distance wider for off-hours / weekend thinness on crypto.

2.5 Known pitfalls this design defends against

- **Overfitting.** Keep to 3-5 core conditions, common parameter values (10/14/20/50/200, RSI 70/30) rather than optimized odd numbers; be suspicious of any backtest claiming a Sharpe above 3.5 or a win rate above 75%.
- **Whipsaw in range-bound markets.** Mitigated by the regime filter and, in Phase 2, a volatility/trend classifier that suppresses crossover-style alerts in chop.
- **Expectancy decay.** A setup that performs well in a trending year can go negative in a choppy one. Phase 3 adds per-setup expectancy tracking so a setup gets paused, not permanently trusted or permanently abandoned, when it turns negative.
- **Behavioral failure.** Pre-committing every level (stop, target, size) at alert time, before any emotional reaction to live price action, is the main defense - which an alerts-first design naturally enforces.

3. Environment Diagnosis (completed)

Diagnosed before any code was written, per the project's own ground rule: no implementation until the actual environment is confirmed, nothing assumed.

Item	Finding	Decision / action taken
Python	3.14.5, Windows 11 (MSC v.1944, 64-bit)	Confirmed compatible with all required packages via smoke test.
Install location	Global: C:\Python314 (70 packages already present, no venv)	Created a project-local .venv to isolate dependencies.
alpaca-py	0.43.4 installed	Correct, current SDK. (alpaca-trade-api, the deprecated legacy package, is absent - correct.)
pandas / numpy	pandas 3.0.3 / numpy 2.4.4	Both very recent major versions. Verified clean import via smoke test.
python-dotenv	1.2.2 installed	Will hold Alpaca + Pushover credentials in .env, gitignored.
pytest	9.0.3 installed	Tests written alongside each module as it's built.
Scheduler	Neither schedule nor APScheduler installed initially	Chose APScheduler - needed for two different cadences (stocks near market close vs. crypto continuous).
TA library	ta-lib and pandas-ta both absent	Deliberately not installing pandas-ta - see decision log below.
Git	2.54.0 available	Repo will be initialized with .gitignore before first commit.

3.1 Decision log (so this isn't re-litigated later)

- **No pandas-ta / no ta-lib.** pandas-ta has a history of breaking on newer numpy/pandas releases (it relies on internals like the old numpy.NaN that get removed in major versions). Given pandas 3.0 and numpy 2.4 are both very new, the fragile dependency isn't worth it for what is only five formulas (SMA, EMA, ATR/Chandelier, RSI, MACD). All indicators are implemented directly in pandas/numpy, unit-tested against known values.
- **APScheduler over schedule.** Needed cron-style flexibility for two different check cadences (stocks vs. 24/7 crypto) without hand-rolling that logic.
- **Dependency versions pinned** to what was already verified to import cleanly together (see requirements.txt in section 4.3), rather than letting pip resolve freely.
- **Smoke-tested before proceeding.** All core imports (pandas, numpy, dotenv, apscheduler, alpaca.trading.client, alpaca.data.historical) confirmed working together in the project venv on Python 3.14.5 before any project code was written.

4. Project Architecture

4.1 Folder structure

```

swing-alert-dashboard/
.venv/                (gitignored)
.env                  (API keys - gitignored, never committed)
.env.example
.gitignore
requirements.txt
README.md
PROJECT_PLAN.pdf      (this document)
src/
  config.py            # env vars, constants: risk %, ATR multipliers, heat cap
  data/
    alpaca_client.py   # bars fetch, stock + crypto
  indicators/
    moving_averages.py # SMA, EMA
    atr.py              # ATR + Chandelier Exit
    momentum.py         # RSI, MACD (confirmation only)
  signals/
    regime_filter.py    # 50/200 MA gate
    entry_signals.py    # pullback, breakout
    exit_signals.py     # structural stop, trailing stop, profit target
  risk/
    position_sizing.py  # 1% rule
    portfolio_heat.py   # 6% cap, BTC-correlation aggregation
  notifications/
    pushover_client.py
  scheduler/
    jobs.py
  main.py
tests/
  test_indicators.py
  test_signals.py
  test_risk.py

```

4.2 Module responsibilities

Module	Responsibility
config.py	Loads .env, defines all tunable constants (risk %, ATR period/multiplier per asset class, heat cap, R:R minimum) in one place.
data/alpaca_client.py	Wraps Alpaca's historical data clients for stocks and crypto; returns clean OHLCV bars.
indicators/*	Pure functions, pandas in / pandas out, no I/O. Each has a matching pytest file with known-value assertions.
signals/regime_filter.py	Boolean gate: is this asset in an uptrend regime right now (above rising 50- and 200-day MA).
signals/entry_signals.py	Detects pullback and breakout setups; only called for assets that already pass the regime filter.
signals/exit_signals.py	Computes structural stop, Chandelier trailing stop, and profit target for an open or proposed position.
risk/position_sizing.py	Converts an entry+stop pair into a position size at the configured risk %.
risk/portfolio_heat.py	Tracks aggregate open risk and BTC-correlated exposure; flags/blocks new alerts over the cap.

Module	Responsibility
notifications/pushover_client.py	Formats and sends the alert payload (symbol, setup, entry, stop, target, size, heat, asset class).
scheduler/jobs.py	APScheduler jobs: stock universe check on one cadence, crypto universe check on a tighter/continuous cadence.
main.py	Wires everything together; the single entry point that's actually run.

4.3 requirements.txt

```
alpaca-py==0.43.4
pandas==3.0.3
numpy==2.4.4
requests==2.34.2
python-dotenv==1.2.2
pytest==9.0.3
apscheduler
```

4.4 .gitignore

```
.venv/
__pycache__/
*.pyc
.env
data_cache/
*.log
.pytest_cache/
```

5. Phased Build Plan

Mirrors the phasing in the original research brief: get the three highest-evidence pillars (regime filter, volatility-based exit, fixed-fractional sizing) working end-to-end before adding any secondary refinement.

5.1 Phase 1 - MVP

#	Task	Status
1	Diagnose environment (Python/venv/packages) before any code	Done
2	Create project venv; install + pin dependencies; smoke-test all core imports	Done
3	Decide architecture / folder structure	Done
4	config.py + .env.example; load and validate Alpaca + Pushover credentials	Next
5	Connectivity check: ping Alpaca paper account, send one test Pushover push	Next
6	Indicators: SMA, EMA, ATR, Chandelier Exit, RSI, MACD (with pytest coverage)	To do
7	Regime filter: above rising 50- and 200-day MA	To do
8	Entry signals: pullback-to-EMA setup, volume breakout setup	To do
9	Exit signals: structural stop, Chandelier trailing stop, 2-3R profit target	To do
10	Position sizing (1% rule) + portfolio heat cap (6%)	To do
11	Pushover alert formatting: symbol, setup, entry, stop, target, size, heat, asset class	To do
12	Scheduler: stock cadence + crypto cadence; wire into main.py	To do
13	Crypto calibration pass: per-asset ATR multiplier (3-5x), smaller sizing, BTC-correlation flag	To do

5.2 Phase 2 - Refinements

- Partial scale-out alerts (50% at 1R, trail remainder).
- Time-based stale-trade alerts (flat after N days).
- RSI/MACD divergence as a confirmation flag layered on existing signals.
- Correlation/heat checks that explicitly flag when a new crypto alert pushes aggregate BTC-correlated exposure over a defined cap.
- Volatility/regime classifier (ADX or ATR%) that labels each asset trending vs. choppy and suppresses crossover-type alerts in chop.

5.3 Phase 3 - Advanced

- Per-setup expectancy tracking from the paper-trade log; auto-flag setups whose rolling expectancy turns negative (pause, don't delete).
- Dynamic ATR multiplier - tighten in calm regimes, widen in volatile ones.
- Any rule change backtested on 5+ years / 100+ trades out-of-sample before promoting it into live alerts.

6. Current Status & Exact Next Step

Completed: environment diagnosed, project venv created, dependencies installed and pinned, all core imports smoke-tested successfully on Python 3.14.5 / Windows 11, architecture and folder structure agreed.

Immediate next step: build `config.py` and `.env.example`, then run a live connectivity check - authenticate against the Alpaca paper account and send one real Pushover test notification - before writing any indicator or signal logic. Credentials are verified working before anything is built on top of them.

Continuation note for any agent picking this up: read section 2 before changing any signal or risk logic - it is the evidence basis the whole system is built on, and parameter choices (ATR period/multiplier, MA lengths, risk %, heat cap) should not be changed without updating this document. Read section 3.1 before adding any new dependency, especially a technical-analysis library. Do not implement any code path that places, modifies, or cancels an order - this system is alerts-only by design, not by current limitation.

Source: distilled and paraphrased from the project's research brief, Rules for a Personal Swing-Trading Alert Dashboard (Stocks + Crypto, Alpaca + Pushover), plus the environment-diagnosis and architecture conversation that followed it.